REQUIREMENTS NOT MET

N/A

PROBLEMS ENCOUNTERED

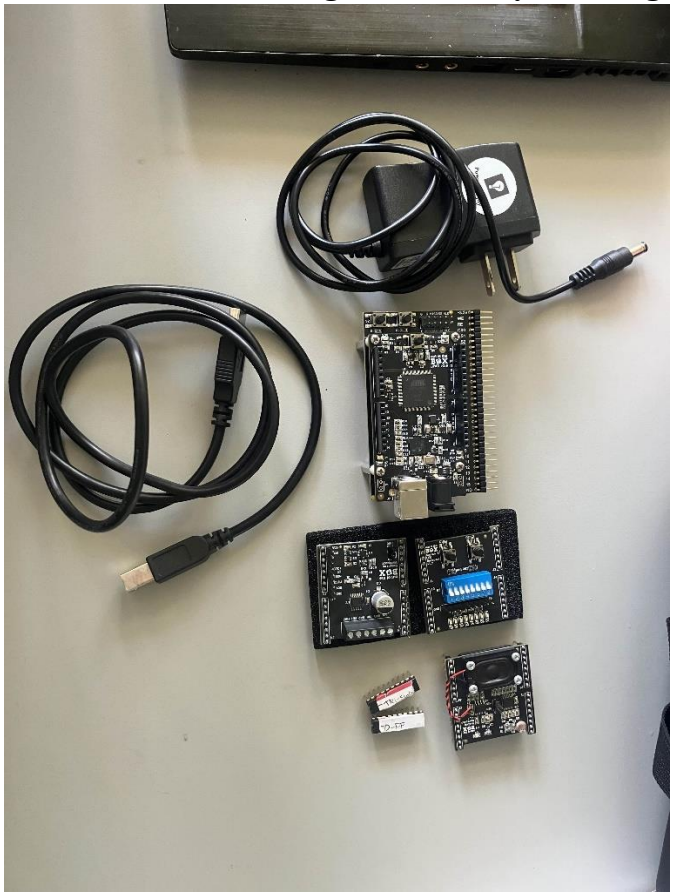
1. I was unable to point to the right data using RAMPZ because I forgot to use ELPM to load from register Z.

FUTURE WORK/APPLICATIONS

1. In this future, I should read the manual more to understand the instructions and understand the use on rampz.
2. I should be aware that in order to check data in the program memory, I need to manually multiply the address by 2 to view it.

PRE-LAB EXERCISES

i. As specified in Lab 0, you should have, upon receiving your kit, verified that it contained all of the parts listed on µPAD v2.0 Parts List (Excel or PDF). If it did not, you should have immediately notified the PI if any components were missing. For documentation purposes (and before any assembly), you should have taken pictures of all of the parts in your kit (each of the PCBs, the chips, etc.). Include these images in your Lab 1 Pre-Lab Report. Your Lab 1 report should also include images of all that you were given in your lab kit.



ii. Which type of memory alignment is used for program memory in the ATxmega128A1U? Byte-alignment, or word-alignment? What about for data memory?

The ATxmega128A1U uses word-alignment for program memory(Flash), that's because the instructions are 16-bits which is one word wide. And the PC addresses each 16-bit word.
For Data memory (SRAM) it uses byte-alignment, that is because the data bus is 8-bits wide, and data is addressed and accessed as individual bytes.

iii. Which assembly instructions can be used to load data indirectly from data memory within XMEGA AU microcontrollers? Which assembly instructions can be used to store data indirectly to data memory?

To load data indirectly, we load them using the pointers that point to data memory location.
LD Rd, X LD Rd, Y LD Rd, Z
They loads one byte from the address pointed to by the X, Y or Z register into general-purpose register.

To store data indirectly, ST X, Rr ST Y, Rr ST Z, Rr
These instructions stores one byte from a general-purpose register Rr to the address pointed by the X,Y or Z.

iv. Which assembly instruction can be used to load data directly from any of the general purpose I/O memory of XMEGA AU microcontrollers? Which assembly instruction can be used to store data directly to any of the I/O memory?

Direct addressing specifies the memory address of the I/O register in the instruction.
Load Directly: IN R16, 0x05 ; read from I/O address 0x05 into R16
LDS R20, 0x0025 ; Load the byte from data memory address 0x0025 into R20

Store Directly to I/O Memory:
OUT 0x1B, R18 ; Write contents of R18 to I/O register at address 0x1B
STS 0x0028, R19 ; Store contents of R19 to memory location 0x0028

The difference between IN/OUT and LDS/STS is that IN/OUT access first 64 I/O register (0x00-0x3F), LDS/STS access entire data space (I/O + SRAM + registers)

v. Which assembler directive places a byte of data in program memory? Which assembler directive allocates space within data memory? Which assembler directives allow you to provide expressions (either constant or variable) with a meaningful name?

.DB or .BYTE they place Bytes in Program Memory use to store constant data.

This instruction allocate space within data memory : .DSEG (data segment)tells the assembler that the following will go into data memory (SRAM).

To provide a expression: .EQU counter = 10 ; Define a constant value 5 (cannot be changed)

.SET COUNTER = 10 ; Define COUNTER as 10 (can be changed later)

vi. Which assembly instructions can be used to read from (flash) program memory? For each instruction, list which registers can be used as an operand.

We can use LPM Rd, Z ; to load a byte from program memory address pointed to by Z-register to Rd.

LPM by itself is load bytes from address in Z into R0 by default

vii. Your Lab 1 report should also include images of all your OOTB PCBs.

Same as i

viii. If you were to use the Memory debug window of Atmel Studio to verify that some datum was correctly stored at address 0xFADE within program memory of the ATxmega128A1U, which address would you specify within the debug window?

The program memory is word-addressed, but the debugger is byte addresses. Therefore we need to convert 0xFADE into byte addresses by multiplying by 2. 0xFADE is 64222 in decimal.
64222 * 2 = 128444, which is 0x1F5DC.

ix. ix. When using the internal SRAM (not EEPROM), which memory locations can be utilized for the data segment (.dseg)? Why?
The memory address that .dseg can occupies range from **0x2000 to 0x3FFF**.

x. Which is the first (i.e., lowest) program memory address (this is an address to the 16-bit wide program memory information) that would require the relevant RAMP register to be changed from its initial value of zero? Why?

The z pointer is being use to access program memory address, and it is a 16-bits register which has 2^16 = 65536 memory location. Since the program memory is word-address, it can access 65536 * 2 = 131072 bytes which is (128KB or 0x0000 to 0xFFFF) of the program memory. To access beyond this, RAMPZ is required starting at address 0x10000.

xi. In the context of pointing an index to a specific program memory address within an XMEGA AU architecture, explain why and how the address value should first be altered. Similarly, in the context of pointing an index to a specific data memory address, explain why the address value should not be altered

The program memory is word-addressed, but the pointers and the underlying hardware work with byte addresses. Each word is two bytes, so to get the correct byte address for a given word, we need to double the word address by left-shifting one bit.

For example, to point to the program memory word address 0x100, I would load the Z-pointer with 0x200

The data memory is byte-addressed, so a given address directly corresponds to a single byte location.

PSEUDOCODE/FLOWCHARTS

```
PseudoCode:

    This is the CONSTANTS

    DEFINE INPUT_TABLE_START = 0xDADD        //Program memory start address

    DEFINE OUTPUT_TABLE_START = 0x3744       // Data memory start address

    DEFINE EOT = null                        // End-of-table marker


    // INITIALIZE

    input_ptr  <- INPUT_TABLE_START           //  Pointer to the address of current input value

    output_ptr <- OUTPUT_TABLE_START          //  Pointer to the address of next output location


    //This is the main program loop

    LOOP:

        current_value <- READ_PROGRAM_MEMORY(input_ptr)  // get next byte from input table


        if ( current_value == EOT ){

            //End of input table reached

            STORE_DATA_MEMORY(output_ptr, EOT)     // store a NULL to terminate output table

            EXIT LOOP                                   // Done processing

        }


       // CONDITION CHECKS

       //case 1 bit7 not set

        if ( BIT7(current_value) == 0 ){        // if bit7 is not set(equal to 0)

            result <- current_value * 2             // Multiply the current value by 2 and store in
the result register

            if ( result ≤ 110 ) {

                result <- result + 12                    //  Add 12

                STORE_DATA_MEMORY(output_ptr, result)   // Save to current location in output table

                output_ptr <- output_ptr + 1             // Move to next output slot

            }


        else {
```

```
            //Case 2: Bit 7 set
            result <- current_value / 2                 // Divide by 2
            if ( result > 70 ) {
                result <- result + 7                     // Add 7 if condition met
                STORE_DATA_MEMORY(output_ptr, result)   // Save to output table
                output_ptr <- output_ptr + 1             // Move to next output slot
            }
        }

        // Move to next input value
        input_ptr <- input_ptr + 1

        // go back to LOOP for next value from the input table
      GO TO loop
    end LOOP with a infinite loop here
```

PROGRAM CODE

```
;*****************************************************************
;  Lab 1, Section 2
;  Name: Peter Feng
;  Class #: 11303
;  PI Name: Grant
;  Description: To filter data stored within a predefined input table
;                                based on a set of given conditions and store
;                                a subset of filtered values into an output table.
;*****************************************************************
;*********************************INCLUDES*********************************
.include "ATxmega128a1udef.inc"
;***********END OF INCLUDES*****************************
;*********************************EQUATES**********************************
; potentially useful expressions
.equ NULL = 0
.equ ThirtySeven = 3*7 + 37/3 - (3-7)  ; 21 + 12 + 4

;***********END OF EQUATES*****************************
;***********MEMORY CONFIGURATION************************
; program memory constants (if necessary)
.cseg               ; indicate a start of program memory
.org 0xDADD  ; start on input table address
IN_TABLE:   ; label marking the start of the input table
;enter all the data in the pdf Table 1
.db       0x21, 0xB4, 0x34,0x38, 0xDA, 0xE4, 0xB8, 0x47, 0x6E, 0x76, 0x2E, 0xD0, 0xCC, 0x32, 0x7D
.db NULL  ; the final byte of the table is null
; label below is used to calculate size of input table
IN_TABLE_END: ; mark the end of the input table
```

```
; data memory allocation (if necessary)
.dseg                    ; indicate of the start of data memory
; initialize the output table starting address
.org 0x3744  ; start of the output table address
OUT_TABLE:               ; mark the start of the out table
.byte (IN_TABLE_END - IN_TABLE) ; use byte to reserve the same number of byte as input table length in this address
;***********END OF MEMORY CONFIGURATION***************

;***********MAIN PROGRAM*******************************
.cseg       ; indicate start of program memory
; configure the reset vector
;          (ignore meaning of "reset vector" for now)
.org 0x0
          rjmp MAIN

; place main program after interrupt vectors
;          (ignore meaning of "interrupt vectors" for now)
.org 0x100 ; the main program starts at 0x100
MAIN:
; point appropriate indices to input/output tables .
;RAMP is needed because 0xDADD * 2 is over 0x10000
; Since IN_TABLE in program memory with word address, *2 to get Byte address
; we need 3 register:  ZH(15..8), ZL(7..0)

;ldi don't work with i/o register, load to r16 first
ldi r16, (IN_TABLE*2)>>16 ; bits 23..16 → 0x01
;refer as CPU_RAMPZ
out CPU_RAMPZ, r16          ; set RAMPZ for LPM
```

```
;load low bits to ZL
        ldi ZL, low(IN_TABLE*2)

;load high 15..8 bits
        ldi ZH, byte2(IN_TABLE*2)

;use Y-pointer for out-table

        ldi YL, low(OUT_TABLE) ;load low byte of the out_table

        ldi YH,high(OUT_TABLE) ;load high byte of the out_table

; loop through input table, performing filtering and storing conditions
LOOP:
        ; load value from input table into an appropriate register

        elpm r16, Z+  ; load what's pointed by Z (in_table) into r16
                                ; increment Z to point to next element in the table

        ; determine if the end of table has been reached (perform general check)
        cpi r16, null ; compare element in the r16 with null
                                ; it's substracting null from r16
                                ;and set Z-flag = 1 if the result is 0

        ; if end of table (EOT) has been reached, i.e., the NULL character was
        ; encountered, the program should branch to the relevant label used to
        ; terminate the program (e.g., DONE)
```

```
            breq DONE ; branch if they are equal
                      ; it's just checking if Z-flag = 1 then jump to DONE
                      ;if Z = 0, execution next instruction

    ; if EOT was not encountered, perform the first specified
    ; overall conditional check on loaded value (CONDITION_1)
CHECK_1:
    ; check if the CONDITION_1 is met (bit 7 of # is cleared);
    ;   if not, branch to FAILED_CHECK1
    sbrc r16,7 ; skip if Bit in Register is clear
                      ;  if bit7(MSB) of r16 is 0 (clear) then skip next instruction
                      ; if bit 7 = 1 , go to next instruction

    rjmp FAILED_CHECK1

    ; since the CONDITION_1 is met, perform the specified operation
    ;   (multiply # by 2, unsigned)
    lsl r16 ; logical shift left to multiple the value in r16 by 2


    ; check if CONDITION_1a is met (result < 111); if so, then
    ;   jump to LESS_THAN_111; else store nothing and go back to LOOP
    cpi r16, 111 ; set C-flag (carry) = 1 if there's a borrow
                      ; there's a borrow if r16 < 111
    brlo LESS_THAN_111 ; branch if lower (unsigned)
                              ; it checks if C-flag = 1, jump to label
                              ; if c-flag = 0, falls through

    rjmp LOOP
```

```
LESS_THAN_111:
    ; add 12 and store the result
    ;we can't use adiw on r16 because it only works on r24..r31
    subi r16, -12 ; substract immediate r16 - (-12) = r16 + 12
    st Y+, r16 ; store the result in r16 to Y(output_table)
                    ; move the pointer to next byte of the table

    rjmp LOOP

FAILED_CHECK1:
    ; since the CONDITION_1 is NOT met (bit 7 of # is set),
    ;   perform the second specified operation
    ;   (divide by 2 [unsigned])
    lsr r16   ;logical shift right to divide by 2


    ; check if CONDITION_2b is met (result >= 71); if so, jump to
    ;   GREATER_EQUAL_71 (and do the next specified operation);
    ;   else store nothing and go back to LOOP

    cpi      r16, 71  ; c = 1 if r16 < 70  (borrow occurred)
    brsh GREATER_EQUAL_71 ;if c = 0 , r16 >= 70(no borrow) -> branch
                                    ; if c = 1 r16 < 70(borrow) -> next instruction

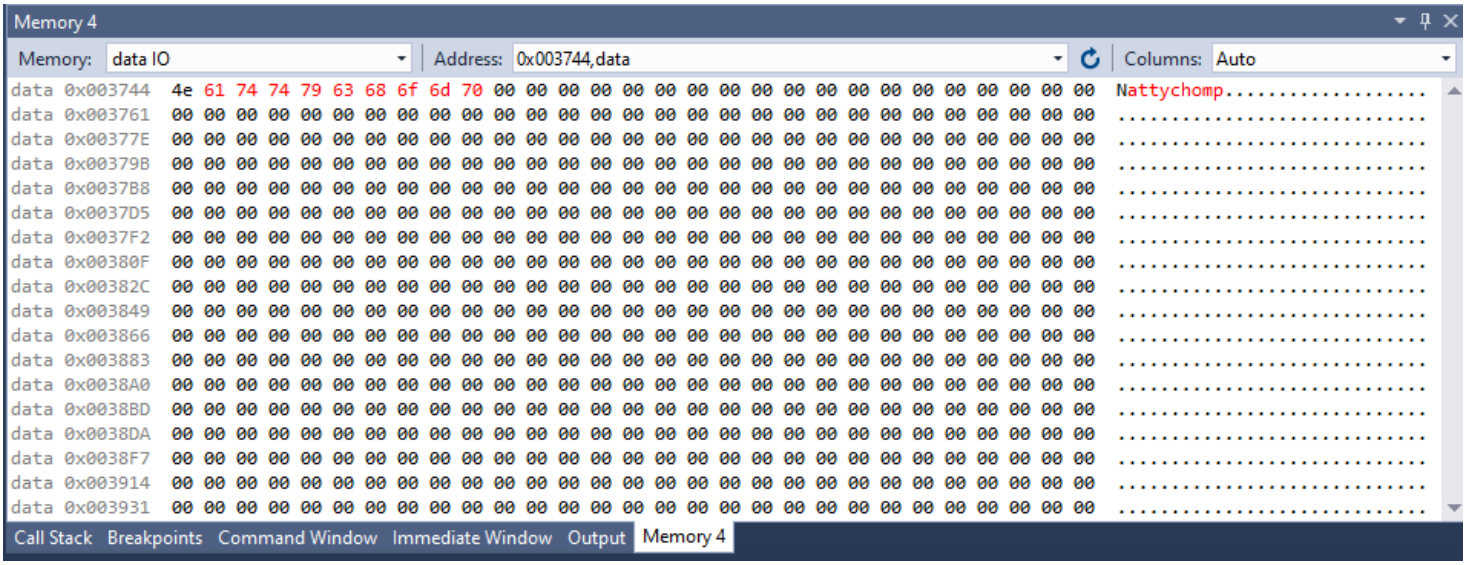

    rjmp LOOP

GREATER_EQUAL_71:
    ; add 7 and store the result
    subi r16, -7
```

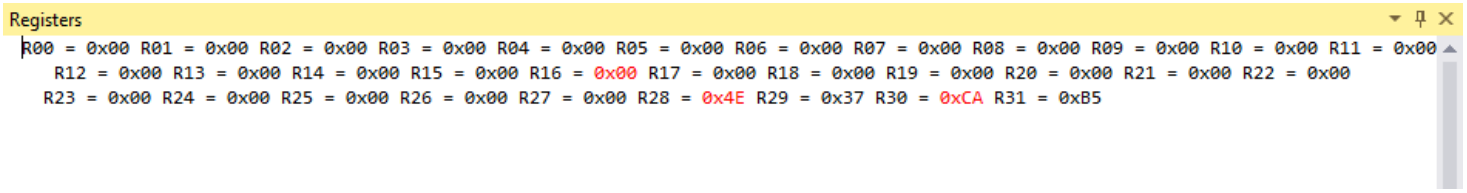
```
        st Y+, r16
        ;go back to LOOP
        rjmp LOOP


; end of program (infinite loop)
DONE:
        rjmp DONE
;***********END OF MAIN PROGRAM **********************
```

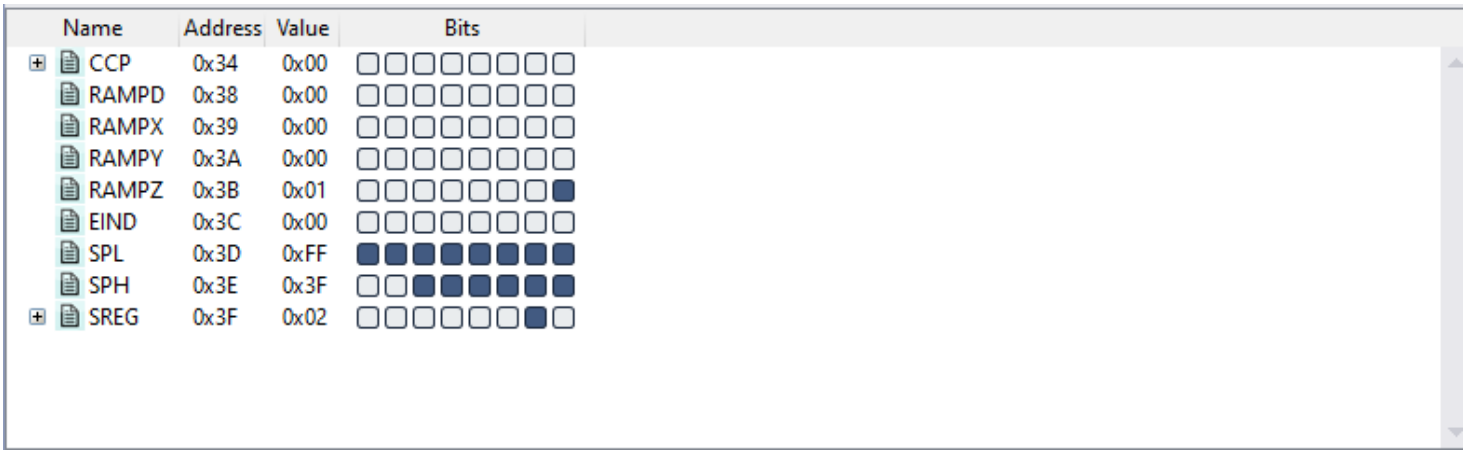
APPENDIX



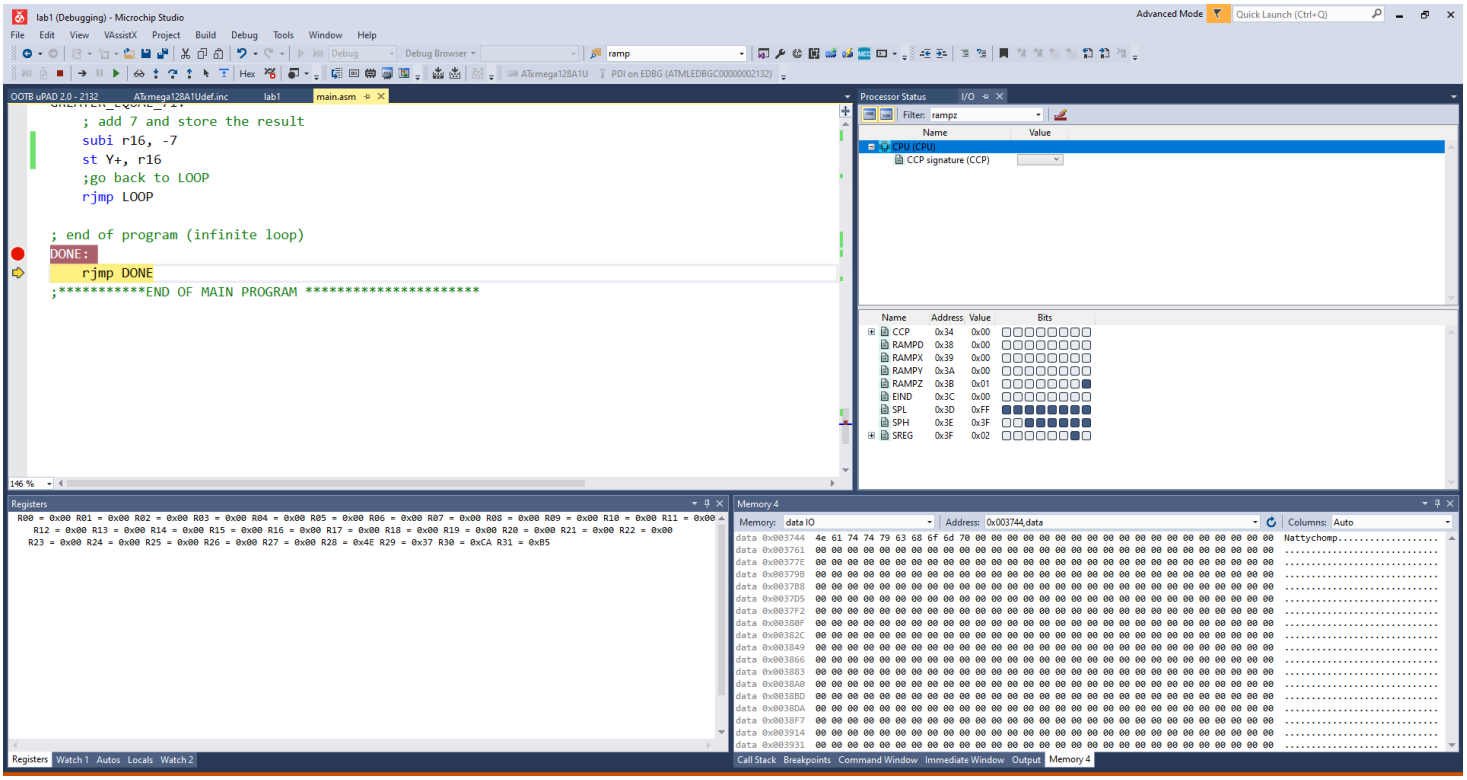
The output table shown at data memory 0x4744.



The registers shown at the end of the program



The I/O registers shown at the end of the program.



This is the result of emulating the program on the uPad.

Local file directory: C:\Users\fengh\Documents\Atmel Studio\7.0\lab1\lab1